CS 4372

Assignment #3

Deep Learning using Keras:

Transfer Learning

Teammates:

Taylor Mitchell (dal501460)

Dylan Ramdhan (dal906790)

Colab Link: CS4372_Assignment_3.ipynb

GitHub Repo Link: https://github.com/dylanramdhan/CS4372/tree/main/Assignment_3

Works Cited

[1] Kaggle Dog Breed Identification

[2] TensorFlow Transfer Learning and Fine-Tuning

[3] TensorFlow Kera Applications MobileNetV2

[4] TensorFlow Image Classification

[5] Stanford CS231n Deep Learning for Computer Vision

1. Dataset

1.1. Before constructing the model, we downloaded the **Dog Breed Identification** dataset from Kaggle[1] using **kagglehub**, and loaded its **label.csv** file, which pairs each image ID with its breed, into a Pandas DataFrame (*Fig. 1.1.1.*).

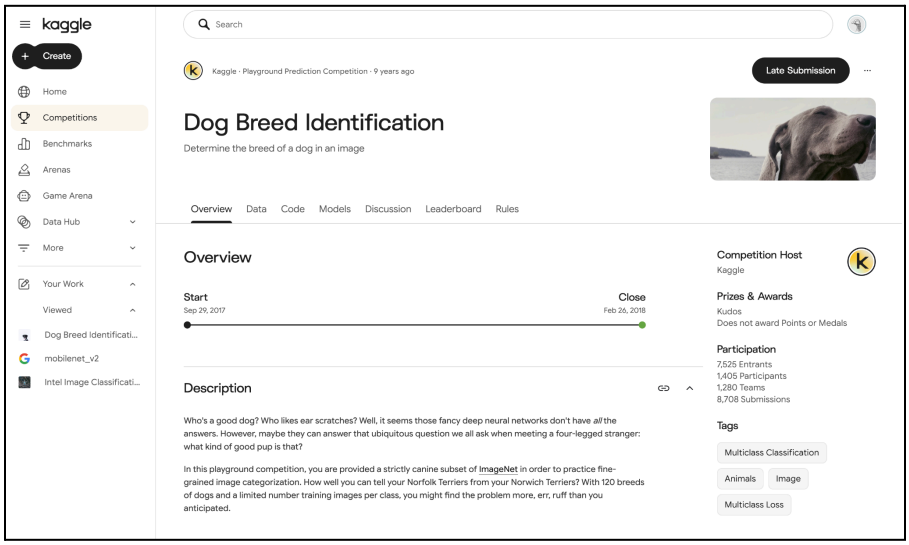


Fig. 1.1.1.

1.1.1. The dataset contains 10,222 labeled images across 120 dog breeds, or about 85 images per breed. The competition also includes a separate test folder, but since those images come without labels, they could not be used to measure accuracy, so only the labeled images were used.

1.2. Since the labeled images did not come per-split, we used scikit-learn's **train_test_split** twice: first holding out 30% of the images, then splitting that 30% in half, giving a total 70% / 15% / 15% split for **training**, **validation**, and **test** (*Fig. 1.2.1.*).

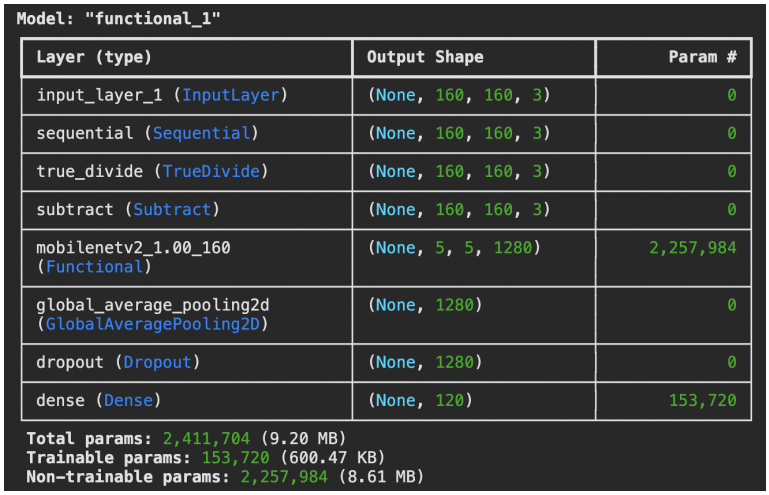
```
Training:    7155 images (70.0%)
Validation:  1533 images (15.0%)
Test:        1534 images (15.0%)
```

Fig. 1.2.1.

1.2.1. This produced 7,155 **training**, 1,533 **validation**, and 1,534 **test** images. Because the split was **stratified** by breed, all 120 breeds appear in each set in the same proportions, each line of *Fig. 1.2.1.* confirm. The **validation set** was used for every tuning decision, while the **test set** was set aside and used only once.

2. Model and Approach

2.1. Rather than training a model from scratch, we used **transfer learning**, building on a model that had already learned general image features. For the pre-trained base we used **MobileNetV2**[3] from Keras Applications, loaded with weights learned on the **ImageNet** dataset. Since its original output layer predicts ImageNet's 1,000 categories, we removed it and added our our **classification head** for the 120 breeds, made of a **GlobalAveragePooling2D** layer, a **Dropout** layer, and a **Dense** output layer, adapted from the TensorFlow transfer learning tutorial[2] (*Fig. 2.1.1*)



Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 160, 160, 3)	0
sequential (Sequential)	(None, 160, 160, 3)	0
true_divide (TrueDivide)	(None, 160, 160, 3)	0
subtract (Subtract)	(None, 160, 160, 3)	0
mobilenetv2_1.00_160 (Functional)	(None, 5, 5, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 120)	153,720

Total params: 2,411,704 (9.20 MB)
Trainable params: 153,720 (600.47 KB)
Non-trainable params: 2,257,984 (8.61 MB)

Fig. 2.1.1.

2.1.1. As shown in *Fig. 2.1.1.*, each 160 x 160 image first passes through MobileNetV2's preprocessing (the **TrueDivide** and **Subtract** layers), which rescales pixel values from *0-255* to the *-1* to *1* range the base was trained on. The base would then output a *5 x 5* grid of *1,280* features, which the pooling layer averages into *1,280* values per image, and the *Dense* layer turns into one score per breed. Of the *2,411,704* total parameters, only the head's *153,720* are trainable, while the base's *2,257,984* stay frozen.

2.2. For **feature extraction**, we froze the entire base and trained only the new head for *10 epochs* using the **Adam** optimizer. We used **SparseCategoricalCrossentropy** as the loss, as in the TensorFlow image classification tutorial[4], since each image's label is a single breed number from *0* to *119*.

2.2.1. Freezing the base keeps the general image features MobileNetV2 already learned from ImageNet, such as edges, textures, and shapes, so only the head has to learn how those features map to breeds. Starting from this baseline, we ran three more experiments, each changing one setting: removing **data augmentation** (random horizontal flips and rotations), lowering the **learning rate**, and raising **dropout**.

2.3. For **fine-tuning**, based on the TensorFlow transfer learning tutorial[2], we started from the best feature extraction model, unfroze the top layers of the base, and kept the lower layers frozen. We then recompiled the model with the **RMSprop** optimizer at a much lower learning rate of 0.00001 and continued training for 10 more epochs (epochs 11-20).

2.3.1. The lower layers of MobileNetV2 detect general features useful for any image, while its top layers detect more specific patterns, so only the top layers were adjusted to fit dog breeds. The small learning rate keeps these adjustments gradual, so the pre-trained weights are refined rather than overwritten. We compared unfreezing the top 54 and top 30 of the base's 154 layers, reloading the saved feature extraction weights before each experiment so both started from the same point.

3. Tuning Table

3.1. We ran 6 experiments in total, changing one setting at a time and recording the training and validation accuracy from the final epoch of each run (*Fig. 3.1.1.*). *Experiments 1-4* are feature extraction runs with the base frozen, while *Experiments 5-6* fine-tune the best feature extraction model (*Experiment 2*).

Experiments	Parameters	Train Accuracy	Validation Accuracy
1	Base = MobileNetV2, frozen base, LR = 0.001, batch size = 32, epochs = 10, dropout = 0.2, with augmentation	76.1%	72.7%
2	Base = MobileNetV2, frozen base, LR = 0.001, batch size = 32, epochs = 10, dropout = 0.2, no augmentation	99.6%	76.7%
3	Base = MobileNetV2, frozen base, LR = 0.0001, batch size = 32, epochs = 10, dropout = 0.2, with augmentation	57.5%	70.2%
4	Base = MobileNetV2, frozen base, LR = 0.001, batch size = 32, epochs = 10, dropout = 0.5, with augmentation	66.3%	74.1%
5	Base = MobileNetV2 (from Exp. 2), top 54 layers unfrozen, LR = 0.00001, RMSprop, batch size = 32, fine-tune epochs = 10, no augmentation	97.6%	73.8%
6	Base = MobileNetV2 (from Exp. 2), top 30 layers unfrozen, LR = 0.00001, RMSprop, batch size = 32, fine-tune epochs = 10, no augmentation	98.5%	75.0%

Fig. 3.1.1.

3.1.1. Each feature extraction experiment changed one setting from the baseline (*Exp. 1*): *Exp. 2* removed augmentation, *Exp. 3* lowered the learning rate, and *Exp. 4* raised dropout. The two fine-tuning experiments differ only in how many of the base's top layers were unfrozen.

4. Best Hyperparameters

4.1. Among the feature extraction experiments, *Experiment 2* (learning rate of *0.001*, dropout of *0.2*, and *no augmentation*) reached the highest validation accuracy at *76.7%*, so it was selected as the starting point for fine-tuning. Compared with the baseline's *72.7%*, removing augmentation helped the most (*+4.0%*) and raising dropout to *0.5* helped slightly (*+1.4%*), while lowering the learning rate at *0.0001* hurt (*-2.5%*) (*Fig. 3.1.1*).

4.1.1. *Experiment 2* also showed the clearing sign of **overfitting**: training accuracy reached *99.6%* while validation leveled off around *76% - 78%* after the third epoch, meaning the head nearly memorized the training images. *Experiment 3* had the opposite problem since its validation accuracy was still rising at *epoch 10*, meaning *10 epochs* were not enough at the lower learning rate. Augmentation likely hurt within *10 epochs* because random flips and rotations of up to 72º (*(RandomRotation(0.2)*: *0.2* of a full turn, *0.2 × 360° = 72°*)) made each training image harder to learn, which kept the baseline's training accuracy at only *76.1%*.

4.2. For fine-tuning, *Experiment 6*, which unfroze the top *30* layers at a learning rate of *0.00001*, reached *75.0%* validation accuracy, higher than the *73.8%* from unfreezing the top *54* layers, making it the final model.

4.2.1. Both fine-tuning experiments reached about *98%* training accuracy but ended slightly below *Experiment 2*'s *76.6%,* a difference of about *26* of the *1,533* validation images. Since *Experiment 2* was already overfitting, unfreezing more layers gave the model more room to memorize the training images, which is why unfreezing *30* layers did better than *54*. We kept the fine-tuned *Experiment 6* as the fine model, since it completes both stages the assignment requires while staying close to the best feature extraction result.

5. Plots

5.1. To view both training sets of the final model, we joined the training history of *Experiment 2* (feature extraction) with that of *Experiment 6* (fine-tuning) into one plot of accuracy and loss over *20 epochs*, with a vertical line marking where fine-tuning began (*Fig. 5.1.1.*).

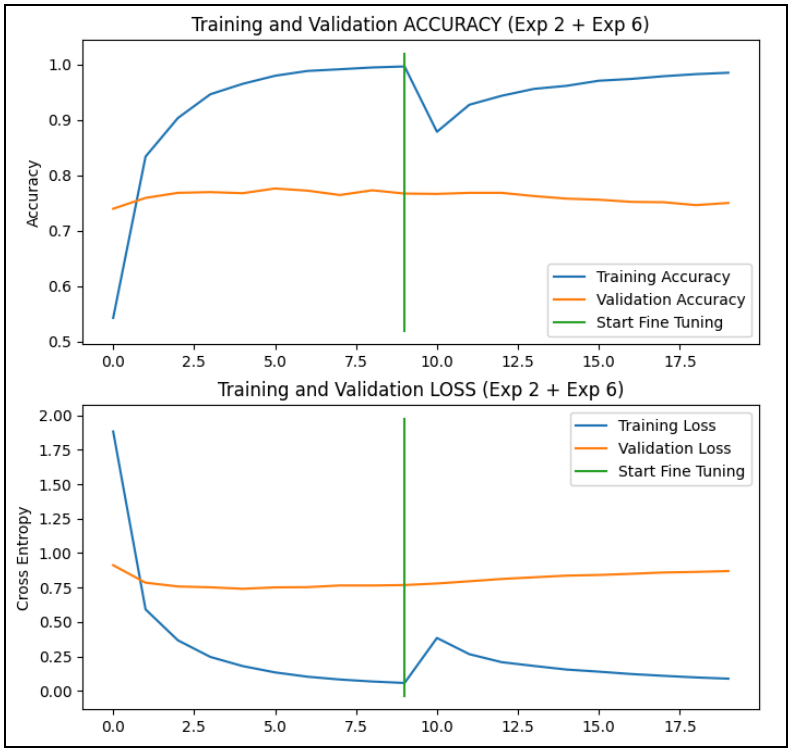


Fig. 5.1.1

5.1.1. During **feature extraction** (*left* of green line), training accuracy climbed quickly from *54.3%* to *99.6%*, while validation accuracy rose only from *74.0%* to a peak of *77.6%* at *epoch 6* before leveling off. The loss curves show the same pattern: training loss fell to nearly 0, while validation loss bottomed out around *epoch 5* and then began to rise slightly, the gap that marks **overfitting** (*Section 4*).

5.1.2. During **fine-tuning** (*right* of green line), training accuracy briefly dripped about *88%* as the newly unfrozen layers began adjusting, then recovered to *98.5%*. Validation accuracy drifted slowly downward to *75.0%*, and validation loss rose steadily, confirming that unfreezing layers mostly helped the model fit the training images further rather than generalize better. The x-axis counts epochs from *0*, so the line at *9* marks the end of *epoch 10*.

6. Test Accuracy

6.1. After all tuning decisions were made using the validation set, we evaluated the final model (*Experiment 2* followed by fine-tuning in *Experiment 6*) **once** on the 1,534 test images it had never seen, reaching a test accuracy of *76.9%* (*Fig. 6.1.1.*).

```
48/48 ━━━━━━━━━━━━━━━━━━━━ 6s 123ms/step - accuracy: 0.7686 - loss: 0.8450
Test accuracy: 0.7686
```

Fig. 6.1.1

6.1.1. This means the model correctly identified the bread in about *1,179* (*0.7686 %* × *1,534 = 1,179*) of the *1,534* test images. The test accuracy is close to the final model's *75.0%* validation accuracy, showing the model performs about as well on new images as it did during turning. It is also far above the roughly *0.8%* (*1 ÷ 120 = 0.0083%*) that random guessing among 120 breeds would score.

7. Sample Predictions

7.1. To show the final model's predictions on individual images, we displayed a 5 x 5 grid of 25 test images, taking the first image from each of the first 25 test batches so the grid covers 25 different breeds. Each image is tilted with its true and predicted breeds, in **blue** if the predictions were correct and **red** if it was wrong (*Fig. 7.1.1.*).



Fig. 7.1.1

7.1.1. The model correctly identified *18* of the *25* images (≈ *72%*), in line with its *76.9%*, test accuracy. Most of the *7* mistakes were between breeds that look alike such as **american_staffordshire_terrier** predicted as a **boxer**, and the **greater_swiss_mountain_dog** predicted as an **entlebucher**, two similar black, white, and tan breeds. Others came from unusual photos, such as a younger looking **great_dane** was predicted as an **italian_greyhound** and a wet, dark **cairn** predicted as a **flat-coated_retriver**.

8. Summary of Results

8.1. Overall, transfer learning worked well for this dataset: by building on MobileNetV2's ImageNet features instead of training from scratch, as CS231n recommends ("download a pretrained model and finetune it on your data")[5], the model reached *76.7%* validation accuracy by training only the new head, and the final fine-tuned model reached *76.7%* test accuracy across *120* breeds. The biggest improvement came from removing data augmentation, while lowering the feature extraction learning rate and fine-tuning did not improve on the best feature extraction result, since the model was already overfitting the training images (*Section 4*).

8.1.1. One approach that did not work was our first fine-tuning attempt, which used a learning-rate of *0.0001* starting from *Experiment 1*. Validation accuracy dropped from *72.2%* to *65.4%* (*top 54 layers*) and *69.1%* (*top 30 layers*), showing the learning rate was large enough to overwrite the pre-trained features. Based on these **validation** results, we lowered the fine-tuning learning rate to *0.00001* and started from *Experiment 2* (the best feature extraction model), and reran the notebook. All results in this report come from that final run.

8.2. We made a few assumptions in the work. We used the image sizes (*160 x 160*), batch size (*32*), and seed (*42*) from the assignment's Appendix example code, and trained every experiment for a fixed 10 epochs so the results could be compared fairly. We also used **MobileNetV2** as the only base model, focusing our experiments on augmentation, learning rate, dropout, and the number of unfrozen layers. Finally, we recorded each experiment's accuracy from its **final** epoch rather than its best, so some results, such as *Experiment 2*'s peak of *77.6%* at *epoch 6*, are slightly lower in the table than their best point during training.